The HetSol Toolkit

Michael Reiter
Institute for Advanced Studies, Vienna and NYU Abu Dhabi

January 7, 2019

Abstract

This is the documentation of the software tool HetSol, which solves DSGE models by linearization and by global projection methods. The special focus of the toolkit are heterogeneous agent models. The toolkit is implemented in the programming language Julia.

Address:

Michael Reiter
Institute for Advanced Studies
Josefstädter Strasse 39
A-1080 Vienna
Austria

e-mail: michael.reiter@ihs.ac.at

1 Introduction

1.1 Outline

"HetSol" is a toolkit to solve DSGE models, with a special focus on heterogeneous agent models. It is implemented in the programming language Julia. Simple models can be solved by the HetSol syntax, without knowing any Julia syntax. The toolkit also allows to use arbitrary Julia language constructs in several places of the model, so as to give a very high degree of generality.

The toolkit generates linearized solutions as well as global nonlinear solutions using projection methods.

1.2 Installation

- Install Julia version 1.0 or higher on your computer from `https://julialang.org/`
- Download the archive "hetsol.tar.gz" at `http://elaine.ihs.ac.at/~mreiter/hetsol.tar.gz`
- Extract the archive "hetsol.tar.gz" in a fresh directory on your computer. This directory with be called "hetsolDir" in the folloing.
- In this directory, edit the file "startup.jl" and change the line

  ```
  @everywhere const hetsolDir = "c:/Users/Michael/Documents/hetsol";
  ```

  to the name of the directory in which you have unpacked hetsol.tar.gz.

  Notice: if you are in Windows, you can use either the "/" for directories, such as

  ```
  hetsolDir = "c:/Users/Michael/Hetsol";
  ```

  or use the double backslash:

  ```
  hetsolDir = "c:\\Users\\Michael\\Hetsol";
  ```

  but not just a single "Ï!!!

  Copy the startup file into the Julia config directory, which is probably the subdirectory ".julia/config" of your home directory, for example

  ```
  "c:/Users/Michael/.julia/config"
  ```

You get your home directory in Julia by typing

```
homedir()
```

- Start Julia and go to the hetsolDir directory. There, execute

```
include("hetsolpack.jl")
```

  which installs the necessary packages.

- Go to the subdirectory "examples" of hetsolDir, and run the command

```
@linsol rbc;
```

  to solve the simple RBC model. Afterwards, you can compute impulse responses with the command

```
(IR,leg) = @irlin(rbc,1,0.01,40,:y,:c,:Inv);
```

  The output "IR" contains the series, the output "leg" contains the legend to be used for a plot. If you have installed gnuplot on your computer, a plot is generated automatically. The command "irlin" is documented in Section sec $sec : irlin$.

With

```
@linsol gali3;
```

you can solve the standard New Keynesian model (Chapter 3 of Gali's book), and with

```
@linsol carlfuerst;
```

you can solve the Carlstrom/Fuerst (1997) model.

To solve the RBC model with global nonlinear methods (projection methods), run

```
include("dorbcnonl.jl")
```

The script "dorbcnonl.jl" contains the necessary steps for solution and analsis of nonlinear models.

For heterogeneous agent models, run

```
@linsolHA changkim;
```

Afterwards, check accuracy by

```
checkAccuracy(changkim);
```

Impulse responses can again be obtained with the irlin command.

2 Example: Basic RBC Model

This example is contained in the distribution as file "rbc.het".

2.1 Some Basics

- Keywords of HetSol are in uppercase format, such as "MODEL", with at least 3 characters. Do not name your variables in such a way, to avoid conflicts with keywords. (Future versions of HetSol may introduce new keywords, but will follow this convention). You can use obvious names such as GDP, because they will never be keywords.
- Names (of variables, parameters, functions etc.) start with a letter, and are followed by an arbitary sequence of letters, digits and the underline ('_') character.
- There are two types of comments.
  1. Anything that follows "#!" will end up as a comment in the generated tex file (at the next parameter declaration, variable declaration or equation).
  2. Anything in a line after a "#" that is not followed by a "!" (to the end of the line) simply gets discarded.

2.2 The complete example program

```
#! capital:
VAR k;
#! labor:
VAR L;
#! output:
```

```
VAR y;
#! consumption:
VAR c;
#! net real interest rate:
VAR r;
#! wage:
VAR w;
#! gross investment:
VAR Inv;
#! productivity (exogenous):
VAR z;
#! productivity shock:
SHOCK epsilon;

#! discount factor:
PAR betta = 0.99;
#! coefficient of risk aversion:
PAR gam = 1;
#! depreciation rate for capital:
PAR delta = 0.025;
#! output share of capital:
PAR alpha = 0.4;
#! weight of leisure in utility:
PAR eta = 1.5;
#! autocorrelation of technology shock:
PAR rho = 0.95;

#! Marginal utility of consumption:
FUNC U_c(c,l) = c^(-gam)*(1-l)^(eta*(1-gam));
#! Marginal utility of leisure:
FUNC U_L(c,l) = eta*c^(1-gam)*(1-l)^(eta*(1-gam)-1);
#!production function:
FUNC F(z,k,l) = exp(z)*k^alpha*l^((1-alpha));
#! Marginal productivity of capital:
FUNC F_k(z,k,l) = alpha*exp(z)*(l/k)^(1-alpha);
#! Marginal productivity of labor:
FUNC F_L(z,k,l) = (1-alpha)*exp(z)*(k/l)^alpha;

MODEL;
```

```
#! Exogenous equation for productivity:
z[t] &= rho*z[t-1] + epsilon[t];
#! Production function
y[t] &= F(z[t],k[t-1],L[t]);
#! Law of motion for capital:
Inv[t] &= k[t] - (1-delta)*k[t-1];
#! Optimal capital input: gross interest rate = marginal productivity of capital:
mpc:   F_k(z[t],k[t-1],L[t]) &= ( r[t] + delta);
#! Optimal labor input: wage = marginal productivity of labor:
w[t] &= F_L(z[t],k[t-1],L[t]);
#! Aggregate resource constraint:
c[t] &=  y[t]  -Inv[t];
#! Household Euler equation:
euler: U_c(c[t],L[t]) &= betta*EXP((1+r[t+1])*U_c(c[t+1],L[t+1])) ;
#! FOC for labor supply:
ls:  w[t]*U_c(c[t],L[t]) / U_L(c[t],L[t]) &= 1;

GUESS;
L  = 0.35 IN [0.01,0.99];
STST;
LIN z;
USING(c=1) LIN(r:euler);
BISECT(k:mpc,0.0001,1000);
LIN y;
LIN Inv;
LIN c;
LIN w;
RESID ls;
```

2.3 Declaring the variables of the model

Let me now explain the different parts of the model file in some detail. The first 8 lines of the program declare the 8 variables in the model, for example

```
#! capital:
VAR k;
```

The keyword is "VAR". One declaration can be used to define several variables, separated by a comma. So the above declaration could also be written as

```
VAR k, L, y, c, r, w, I, z;
```

or

```
VAR k, L, y;
VAR c, r, w, I, z;
```

The command is always ended by a semicolon.

Talking about variables, it is important to distinguish between what is a variable in the sense of computer science (basically any object in the computer program) and economic variables (something that can vary from one time period to the next.) A parameter is not a variable in the economic sense, although it is a variable in the computer program. In the following, I use the term "variable" in the economic sense.

2.4 Declaration of Shocks

The standard RBC model has only one shock, declared with the keyword "SHOCK":

```
SHOCK epsilon;
```

Otherwise, the syntax of SHOCK is like that of VAR. There can be an arbitrary number of shocks.

2.5 Setting Parameters

Parameters are declared by the PAR command. A value is attached by assignment with '=', for example

```
#! discount factor:
PAR beta = 0.99;
```

Again, one can declare and assign several parameter on one line, but it is probably better style to use a separate line for each parameter.

2.6 Declaring Functions

The keyword is FUNC; it is used to define one-line functions, for example

```
#! Marginal utility of consumption:
FUNC U_c(c,l) = 1/c;
```

2.7 The Equations of the Model

The following code defines the model equations. Start the equation section by

```
MODEL;
```

Since we have 8 variables per period, we need 8 equations in the model:

```
#! Exogenous equation for productivity:
z[t] := rho*z[t-1] + epsilon[t];
#! Production function
y[t] := F(z[t],k[t-1],L[t]);
#! Law of motion for capital:
I[t] := k[t] - (1-delta)*k[t-1];
#! Optimal capital input: gross interest rate = marginal productivity of capital:
mpc:   F_k(z[t],k[t-1],L[t]) &= ( r[t] + delta);
#! Optimal labor input: wage = marginal productivity of labor:
w[t] := F_L(z[t],k[t-1],L[t]);
#! Aggregate resource constraint:
c[t] :=  y[t]  -I[t];
#! Household Euler equation:
euler:   1 &= beta*EXP((1+r[t+1])*U_c(c[t+1],L[t+1])) / U_c(c[t],L[t]);
#! FOC for labor supply:
ls:  w[t]*U_c(c[t],L[t]) / U_L(c[t],L[t]) &= 1;
```

Comments:

1. The equations can be written in arbitrary order.

2. Each variable has a time index, given in brackets, which can either [t], [t+1], or [t-1].

   Notice that each equation can hold either

   - variables of time t only
   - variables of time t and $t+1$
   - variables of time $t-1$ and t.

   For the linearized solution, it is not necessary to put an expectation operator. Every variable with time index $t+1$ is understood as referring to the future, so it is assumed that we have to take expectations.

   If you wish, you can explicitly write the expectation operator "EXP", for example

   ```
   euler:   U_c(c[t],L[t]) &= beta*EXP((1+r[t+1])*U_c(c[t+1],L[t+1]));
   ```

   The difference is that then the expectation operator appears in the generated Latex file.

   For computing nonlinear solutions, the expectations operator must be set explicitly.

3. Shocks must have time index [t].

4. The expression

```
x[stst]
```

refers to the steady state value of the variable x. For example, a Taylor rule could be written as

```
log(R(t)) &= Rbar + gamma*log(pi(t)/pi[stst]);
```

5. Not just each variable, but also each equation has a name, which will be referred to later in the solution.

   There are two different ways to write an equation:

   (a) The name of the equation is given explicitly. In this case, the equation is started by the name of the equation, followed by a colon:

   ```
   mpc:   F_k(z[t],k[t-1],L[t]) &= ( r[t] + delta);
   ```

   The name of this equation is "mpc".

   (b) If there is exactly one variable on the left hand side of the equality, such as

   ```
   y[t] &= F(z[t],k[t-1],L[t]);
   ```

   then the name of this variable is also the name of the equation. This is equivalent to

   ```
   y: y[t] &= F(z[t],k[t-1],L[t]);
   ```

   If an equation serves to determine one variable, it is convenient if the equation share the name with the variable, as we will see when solving for the steady state.

2.8 Solving for the deterministic steady state

In our simple example, we solve for the steady state by one-dimensional root finding over the variable labor. We guess it is in the interval (0.01:0.99):

```
GUESS;
L  = 0.35 IN [0.01,0.99];
```

The keyword "GUESS" opens the section of "guesses", that is the initial values for the numerical equation solving in the steady state. We look for L in the (0.01:0.99) with starting value L = 0.35. There can be arbitrarily many guesses. Fewer guesses mean better chances

for convergence. If there is only one guess, the solution will be found for sure if there is one in the interval given in brackets.

Next we will start the steady state computations by the command

```
STST;
```

First we can solve the equation named z for the steady state of the variable z. This equation is linear in z. This can be done by the simple command

```
LIN z;
```

More generally, if the equation name "equ" is solved linearly for the variable "var", the command is

```
LIN var:equ;
```

There is also a short form of this command (solving linearly for on equation in one variable); simply type

```
!var:equ;
```

or

```
!var;
```

if the name of the equation equals the name of the variable.

In the next line, we linearly solve the equation named "euler" for the variable r. Notice that the variable c appears in the Euler equation, but its value has not yet been determined. This does not matter, because in steady state, the variable cancels on both sides of the equation. So we can set an arbitrary value for the variable c, which we do by the USING instruction. Although the exact value does not matter, it has to be positive, to avoid an arithmetic error in the utility function.

```
USING(c=1) LIN(r:euler);
```

With the USING command, we set the value of a variable for the purpose of solving one equation, afterwards this value is unset again.

Next we solve the equation "mpc" for capital k. This is a nonlinear equation in k, and we must solve it nonlinearly within an interval. This is done by the line

```
BISECT(k:mpc,0.0001,1000);
```

This searches for k in the interval $(0.0001, 1000)$. The interval can be given not just as numerical constants, but can contain other variables, if they are already known at the current stage, for example

```
BISECT(k:mpc,0.0001*L[stst],1000*L[stst]);
```

Notice that variables must be referred to as steady states, since this is a steady state computation.

The next four lines solve again one equation for one variable each:

```
LIN y;
LIN Inv;
LIN c;
LIN w;
```

So far we have used 7 equations to solve for 7 variables. There is one equation left that has not been used to compute any steady state values, namely the FOC for labor supply. This equation can be used to iteratively search over the for the value of the guessed variable L. It is declared as the residual equation:

```
RESID ls;
```

This completes the model file.

3 Defining a simple model, more generally

A simple het-file has the following structure:

1. Declaration section
2. Model section
3. Steady state section
4. Nonlinear solution section (only for nonlinear solution)

3.1 Declaration section

In the declaration section, there are three types of declaration: parameters, variables (including shocks) and simple (one-line) functions and macros. They can be freely mixed, with the exception that shocks must be defined *after* all the variables havem been defined.

Any line in the declaration section that does not begin with a declaration keyword is written unchanged into the file where parameters are initialized. This allows complete freedom in the construction of parameter values, as long as these commands are valid Julia code.

3.1.1 Parameter declarations

Consists of a sequence of declarations. Each declaration starts with a keywords, and is finished by a semicolon. Admissible keywords are:

- PAR
- PARCALI
- INT
- BOOL

The general syntax is the following:

```
TYPE p = value;
TYPE{d} p = value;
TYPE{d} p;
```

where TYPE is any of the above keywords. If the parameter is a scalar, it must be initialized by a value. If the parameter is an array, it can either be initialized immediately, or just declared, and later then given a value. Of course, the value that is given to an array must be of the right dimension, otherwise Julia will cast an error. For value, there can be the keyword "GLOBAL", which means that the value is taken from the global scope (at initialization; may be changed later). Notize that the value is *copied* from the global workspace at initialization. Later changes to an array in the global workspace will not affect the parameter.

For "d", any integer value greater than zero is allowed. For example, d=2 means that the parameter is a two-dimensional array of the appropriate type.

The syntax for calibrated parameters is somewhat different:

```
PARCALI p;
PARCALI{1} p(n)
PARCALI{2} p(n1,n2)
PARCALI{3} p(n1,n2,n3)
```

Calibrated parameters behave similarly to variables: they are not initialized, but their value will be determined later. Therefore, their size (not just their dimension) must be specified when declared.

Notice that the notation for arrays mimics Julia notation. In

```
PARCALI{2} p(n1,n2)
```

the dimension in braces is similar to Julia's

```
p::Array{Float64,2}
```

while the size in parentheses is similar to

```
p = Array{Float64}(n1,n2)
```

Notice that several declarations can be combined on one line, e.g.

```
PAR beta=0.3,gamma=2.3;
PAR{1} zeta,phi;
```

3.1.2 Variable declarations

There are two types of variables: endogenous variables, introduced by the keyword "VAR", and i.i.d. shocks, introduced by the keyword "SHOCK". The general syntax for declarations is

```
TYPE v;
TYPE{1} v(n);
TYPE{2} v(n1,n2);
```

where TYPE is either "VAR" or "SHOCK". In the case of arrays (dimension "1") or matrices (dimension "2") of variables, the size must be given, as was the case for calibrated parameters. In special circumstances, the size of arrays of variables may be changed later in the steady state section. This can be the case for heterogeneous agent models, where the individual problem is solved by dynamic programming on a grid, where the grid size can be adapted endogenously.

Again, several declarations can be combined on one line, e.g.

```
VAR{1} x(T), v(T);
```

3.1.3 Function Declarations

There are two types of user-defined functions:

1. Short (one-line) functions, introduced by the keyword FUNC:

   ```
   FUNC F(z,k,l) = z*pow(k,alpha)*pow(l,(1-alpha));
   ```

2. Long functions, that are directly interpreted as Julia code, cf. Section **??**.

Short functions can be symbolically differentiated. The command

```
FUNCDIFF F k F_k;
```

defines the function F_k(z,k,l) as the partial deriative of F(z,k,l) with respect to k.

3.2 Model section

The model section of a simple model only consists of equations. A model equation extends over several line, if the first line is recognizable as a model equation. This requires that at least one of the following items appear on this line:

- a time index [t], [t+1] or [t-1]
- the sequence "&="
- an equation name specifier such as "xyz:", "xyz[i]:", "xyz[i,j]:" etc.
- the keyword CALI

If the first line of an equation is recognized, the parser continues until it finds a semicolon to end the equation.

3.3 Steady state section

Steady state initialization ("Guesses")

The aim of the steady state section is to compute the deterministic steady state in a robust way, so that it does not fail even if many different parameter combinations are used. The most important feature to get robustness is to reduce the big system of nonlinear equations (n equations in n variables) to a nonlinear system of equations of much lower dimnension in the outer loop. In the inner loop, all other equations are solved for in a way that cannot fail.

Example above: in the RBC model, we only "guess" one variablek namely labor supply. the command

```
L  = 0.35 IN [0.01,0.99];
```

means that L is within the interval $(0.01, 0.90)$, with a starting guess of 0.35. An arbitrary number of guess can be given in the section started by the keyword "GUESS".

Based on this initial guess, all the other variables are solved for recursively in a safe fashion, by the commands "LIN", "BISECT" and "KNOW"

LIN command

Examples:

```
LIN(var:eq);
LIN(var1:eq1,var2:eq2,var3:eq3,var4:eq4,var5:eq5);
```

The LIN command takes as arguments an arbitary number of variable:equation pairs. If the name of the equation is equal to the name of the variable, the equation name can be dropped, such as in the example

```
LIN(z);
```

It solves for the variable name "z", using the equation named "z". If the LIN command is not nested within other commands (see below), it is possible to drop the parenthesis and write

```
LIN var:eq;
LIN var1:eq1,var2:eq2,var3:eq3,var4:eq4,var5:eq5;
```

BISECT command

The command BISECT is used to solve one nonlinear equation in one variable. For example,

```
BISECT(k:mpc,0.0001,1000);
```

solve the equation mpc for the capital stock k, in the interval (0.0001,1000). As always, the equation name can be skipped if it equals the variable name.

There is a second, more powerful version of the BISECT command. Take the following stylized example:

$$\begin{aligned} x[t] &= x[t]^2 + y[t]^2 + \sin(z[t]) \\ y[t] &= x[t] + 2z[t] \\ z[t] &= 3x[t] + 2y[t] \end{aligned} \tag{1}$$

The equation x is nonlinear in x, y, z, but conditional on x, equations y and z can be solved *linearly* for y and z. This situation can be handled by

```
BISECT(x,-1000,1000,LIN(y,z));
```

It solves for x using bisection, whereby it solves linearly for y and z for any given value of x.

If the nonlinear function gives values of opposite sign at the lower and upper boundary of the interval, it will find a solution of the equation, supposing continuity of this function. Notice that BISECT does not check for uniqueness of a solution.

KNOW command

If we can solve analytically for the value of the variable var as a function of the variables defined so far, we can use the command

```
KNOW(var:equ=expr);
```

This procedure is save in the sense that HetSol checks whether the expression expr really solves the equation equ for the variable var.

For example, rather than using bisection above, we could solve for the capital stock with the command

```
KNOW(k:mpc=pow(l/k,1-alpha) = L[stst] * (alpha/ ( r[stst] + delta))^(1/1-alpha));
```

Notice that in the expression on the rhs of the equation, steady state variables must be explicitly subscripted by "[stst]".

4 Solving the model

Steps in the solution:

1. Preprocessing (done only once unless model is changed). This is done with the command

   ```
   defmodel(hetfilename)
   ```

   where hetfilename is a string with the name of the hetfile (without filename extension).

   At this stage, information about size of model (number of cohorts in OLG model, number of assets etc.) can be parametric. It is only necessary to set the global variables that affect preprecossing (conditional compilation, cf. Section **??**).

2. Solving for the deterministic steady state of the model

3. Computing the linearized solution

4. Computing the nonlinear solution (if nonlinear section in het-file is specified).

Steps 1–3 are achieved by the single macro command

```
@linsol <hetfilename>
```

For example

```
@linsol rbc
```

computes the linearized solution of the model described in the file "rbc.het". The solution is stored in the structure called "rbc". To store it in a different structure, for example "rbc1", type

```
rbc1 = @linsol rbc
```

The linear solution generates the matrices A and B, which give the model solution the form

$$x_t = Ax_{t-1} + B\epsilon_t \tag{2}$$

The structure rbc generated by @linsol has the components furthermoer contains the components

- pars: parameter values
- vstst: steady state values of all variables
- equNames: a dictionary giving the equation names and corresponding numbers
- varNames: a dictionary giving the variable names and corresponding numbers
- guess: the equilibrium value of the "guesses" for the steady state solution.

4.1 Preprocessing

To do only the preprocessing, you can type

```
defmodel("rbc")
```

Before doing that, HetSol must have been initialized, which can be done by typing

```
@hetsol
```

Preprocessing generates a series of Julia files (extension ".jl") which are defining the model. Most important for the linearized solution are the files "linsol.jl" and "types.jl". Furthermore, it sets parameters to the values defined in the model file. and packs all the parameters into the global variable "_pars".

5 Saving results and exporting results to other formats

5.1 Saving and loading a linear solution

If a model solution has been obtained, for example by

```
@linsol rbc;
```

the resulting solution structure (of type "LinSol") can be saved to the file "rbc.bin" with the command

```
save("rbc.bin",rbc)
```

The file extension is arbitrary, the ".bin" here indicates that this is a binary file.

To load the solution of a model, this model must first be defined (although not solved). The sequence of commands

```
@hetsol
defmodel("rbc")
sol = loadLinSol("rbc.bin")
```

first initializes HetSol, then defines the model "rbc", and then loads the solution saved above and puts it into the structure "sol". If several models have already been defined in the current Julia session, the command

```
sol = loadLinSol("rbc.bin",id)
```

loads the model with the identification number "id". (Notice that this id cannot be saved in the binary file, because a new number is assigned to a model in every Julia session.)

5.2 Exporting to Dynare .mod file

After solving a model with the toolkit (linearly), the command

```
writeDynare()
```

generates a .mod file for solving this model with Dynare.

- It uses the value 0.01 for the standard deviation of all shocks. If necessary, this has to be changed.
- The last line in the file tells Dynare what to do, and has probably to be changed by the user.
- It gives the steady state values of all variables as parameters to Dynare, so that it is trivial for Dynare to find the steady state.

5.3 Exporting results for use in matlab

The command

```
write2mat("myout",modelname)
```

writes the solution of the model "modelname" to the matlab readable file "myout.mat". It defines the following variables:

```
A:          Solution matrix
B:          Solution matrix
guess:      Vector of staedy state guess
ie:         Structure with equation names
iv:         Structure with variable names
vstst:      Structure with steady state values
```

6 Advanced Features

6.1 Control variables

The global variable "_linsol" controls the computation of the linearized solution. It has (at least) the following components:

- crit (Float64): threshold for whether eigenvalue is considered stable
- isZero (Float64): threshold for setting very small numbers to zero in matrices A,B
- nDoQZ (Int): number of variables until which the solution is computed via QZ decomposition;

  for n>nDoQZ, the linear iteration is used
- loadStSt (Int): if greater than zero, load the guess for the steady state from file "xstst.bmt"
- tolResidStSt (Float64): tolerance level for steady state residals; if any residual is greater, an error is thrown

6.2 Calibrated Parameters

A parameter may not be fixed exogenously, but by matching a steady state target. We can change the above example by using the depreciation rate δ to set the capital output ratio in steady state equal to some parameter K_Y. We declare parameters by

```
PAR K_Y = 12;
PARCALI delta;
```

and include in the model section the equation

```
CALI k[t] &= K_Y * y[t];
```

The modifier CALI indicates that this equation is only used for calibration (that means, in steady state), not in deviations from steady state.

6.3 Conditional preprocessing

It is possible to generate several versions of a model from the same het-file by preprocessing commands such as "!!if".

Preprocessor commands start at the beginning of the line

```
!!if exogLabor
  labsupply:  L[t] &= 1/3;
```

```
!!else
  labsupply:  w[t]*U_c(c[t],L[t]) &= U_L(c[t],L[t]);
!!end
```

The expression following "!!if" to the end of the line must evaluate to a boolean value (true or false) at the time of preprocessing. Variables (here the variable "exogLabor" are evaluated at the Main module, and must be available as global variables at the time of preprocessing. They have nothing to do with the parameters of the model, which are not yet available at preprocessing. If a preprocessing variable should also serve as a model parameter, the parameter should be declared as global in the hetfile:

```
PAR exogLabor = GLOBAL;
```

This means that, at the time of parameter initialization (step 2 in Section), the model parameter exogLabor is assigned the value of that variable in the global namespace of Main.

6.4 General functions

[TO BE COMPLETED]

7 Model analysis: impulse response functions

Once the linearized solution is computed and saved in a structure, impulse responses can be computed with the Julia function irlin.

```
rbclin = @linsol rbcnonl
```

Syntax for irlin:

```
IR = irlin(rbclin,iShock,sizeShock,T,args...)
IR = irlin(rbclin,nameShock,sizeShock,T,args...)
IR = irlin(rbclin,nameShock,indxShock,sizeShock,T,args...)
```

where

- "model" is the name of the model solution, obtained for example by

  ```
  model = @dsge modelfile
  ```

- "iShock" is an integer, giving the number of the shock for which the IR is computed.

  If iShock<0, it gives the IR to a change in the initial value of the variable whose number is the absolute value of iShock. For example,

```
IR = irlin(rbclin,-5,0.01,...
```

is the path starting from the steady state with variable 5 increased by 0.01. Notice that this just gives the steady state if variable 5 is not a state varable.

- "nameShock", which is a symbol, gives the shock not by its number but by its name. Example:

```
IR = irlin(rbclin,:epsilon,0.01,40,args...)
```

- If "shockName" indicates an vector-valued shock, then it must be followed by an integer indicating the element of the shock. Example:

```
IR = irlin(rbclin,:epsilon,3,0.01,40,args...)
```

is the IR to the shock "epsilon[3]".

- sizeShock is a floating point number indicating the size of the initial shock.
- T is an integer giving the length of the IR.

The variable "args..." stands for all the other remaining arguments, indicating the variables for which to plot the IR, and further commands. Arguments can be of the following types:

- Symbol: indicating the variables to plot.

```
IR = irlin(rbclin,:epsilon,0.01,40,:Y,:wage)
```

plots output and wage for the model "rbc".

- Integer: only allowed if following a symbol that refers to an array variable, giving the index of the variable. Example:

```
IR = irlin(rbclin,:epsilon,0.01,40,:p,3,:p,5)
```

plots IR of p[3] and p[5].

- Range: same as integer, giving the range of indices of a variable. Example:

```
IR = irlin(rbclin,:epsilon,0.01,40,:p,1:10)
```

plots IR of p[1] to p[10].

```
IR = irlin(rbclin,:epsilon,0.01,40,:p,2:2:10)
```

plots IR of the p with even indices between 2 and 10.

- Floating point numbers: this scales the preciding variable.

```
IR = irlin(rbclin,:epsilon,0.01,40,:Y,3.0,:wage,2.3)
```

multiplies Y by 3, and wage by 2.3

- By default, output is given as a deviation from steady state, expressed as fraction of the steady state value (unless the steady state is 0, in which case the output is not normalized). To change this, put the command "dev" in front of the variables, such as

```
IR = irlin(rbclin,:epsilon,0.01,40,"dev",:Y,:wage)
```

so that the deviation from steady state is given, not normalized by the steady state value.

The command "perc" switches back to percent of steady state, the default:

```
IR = irlin(rbclin,:epsilon,0.01,40,"dev",:Y,"perc",:wage)
```

Here the wage is normalized by the steady state, while output Y is not.

To give the original value of the variable (including the steady state, not in deviations), put "abs" in front of the variables:

```
IR = irlin(rbclin,:epsilon,0.01,40,"abs",:Y,:wage)
```

Both variables are reported as original values.

- Function: unary functions are allowed, for example log:

```
IR = irlin(rbclin,:epsilon,"abs",:Y,log)
```

shows the log of Y; the "abs" is necessary to first add the steady state, to avoid negative numbers.

The function diff is special:

```
IR = irlin(rbclin,:epsilon,0.01,40,:Y,diff)
```

plots the first difference of Y (again in percentage of steady state, if not otherwise indicated).

- Plotting commands are given as strings:

- "file=xyz" saves the plot as xyz.pdf
- "x=xyz" labels the x-axis as "xyz"
- "y=xyz" labels the y-axis as "xyz"
- "title=xyz" makes "xyz" the plot title
- "leg=xyz" makes "xyz" the legend key of the preceding variable

Example:

```
IR = irlin(rbclin,:epsilon,0.01,40,:Y,"leg=GDP",:wage,"x=Quarters","title=My graph")
```

Order of variables:

- The commands "abs", "dev" and "perc" refer to all the variables that follow the command (until a new command of these three is given)
- All commands that apply to a single variable refer to the preceeding variable. They can be combined ("postfix notation"), for example:

  ```
  IR = irlin(rbclin,:epsilon,0.01,40,:Y,diff,5.0)
  ```

  plots 5 times the first difference of Y.

Output: the output variable (above always called "IR") has 2 components:

1. IR[1] contains the IRs, one column per variaable.
2. IR[2] is an array of strings, containing the legend keys of each variable.

8 Heterogenous agent models

8.1 A class of HA models

- The agent problem has three state variables:

  1. An endogenous continuous state (for example financial assets)
  2. An endogenous discrete state. Examples:
     - employed versus not employed
     - skilled versus unskilled (if this is a consequence of past action)
     - owned housing with a finite number of possible house sizes
  3. An exogenous discrete state, such as exogenous labor productivity. The dynamics are modeled as a finite Markov chain.

  Of course, several discrete states can always be combined into For example, two employment states and three skill groups can be combined into a discrete state of six possible realizations.

- The agent problem has two decision variables:

  1. The endogenous continuous choice variable, which is taken for reasons of computational efficiency as the end-of-period continuous state (such as saving, end-of-period capital).
  2. An endogenous discrete choice, such as working versus not working.

8.2 An example program

The first block are parameter declarations, as for any other model.

```
# PARAMETER DECLARATIONS:
# TECHNOLOGY:
PAR alpha = 0.36;  # coefficient of capital in prod.func.
PAR rho_aggr = 0.95;   # correlation TFP
PAR delta = 0.025; # depreciation rate
# HOUSEHOLD:
PAR beta = 0.98267; # discount factor
PAR hbar = 1/3;  # hours worked if working
PAR{1} Hours = [0.0,hbar]; # put into a vector
PAR rho = 0.929;  # correlation individual productivity
PAR sigma = 0.227; # stdev productivity
PAR gam = 0.4; # utility parameter in Chang/Kim
PAR B = 166.3; # utility parameter in Chang/Kim
```

```
# set eta such that: eta*H = B*H^(1+1/gamma)/(1+1/gamma)
PAR eta = B*hbar^(1/gam)/(1+1/gam); # disutility of working
PAR lowerBoundK = -2.0; # borrowing constraint
# grid sizes for household dynamic problem:
INT nY = 17; # number of individual productivity states
INT nGridV = 400; # size capital grid for value function; grid will be set in StSt section
INT nGridD = 1001; # size capital grid for distribution; grid will be set in StSt section
(TP,logY)= markovappr(rho,sigma,2.5,nY);
PAR{2} TransProdtty = TP;
PAR{1} indProdtty = exp.(logY);
```

Then come variable declarations and short functions. The value function for the household problem ("V") is a three dimensional array of variables. The first dimension is the number of points in the grid of capital (the endogenous household state). The second dimension is the number of the discrete endogenous states, which is 1 here (there is no discrete endogenous state variables for the household). The third dimension is the number of the discrete exogenous states, in this case the number of possible realizations of household productivity.

The cross-sectional distribution of household states ("D") is also a three dimensional array of variables. The second and third dimension coincide with that of the value function. The first dimension is the number of intervals of the grid of capital. It differs from the first dimension of V for two reasons. First, the size of the distribution is one less than the number of grid points, because the distribution is represented by the fraction of agents in an interval between grid points. Second, the capital grid to represesnt the distribution can differ from the grid on which the value function is defined.

```
##############################################################################
# VARIABLE AND PARAMETER DECLARATIONS:
##############################################################################
# cross-sectional distribution of households across all states
# dimension is one less than grid points (= intervals between grid points)
VAR{3} D(nGridD-1,1,nY);
VAR z;  # aggregate productivity
VAR{3} V(nGridV,1,nY); # value function
VAR r;  # real interest rate
VAR Inv;
VAR C;
VAR wage; # wage
VAR KLratio;  # capital-labor ratio
VAR K;  # aggregate capital
VAR KEnd;  # aggregate capital at end of period
```

```
VAR H;  # aggregate hours
VAR L;  # aggregate labor input
VAR GDP;  # aggregate labor input
SHOCK epsilon;  # exogenous i.i.d. shock

##########################################################################
# SHORT FUNCTIONS:
##########################################################################
FUNC F(z,k,l) = exp(z)*pow(k,alpha)*pow(l,(1-alpha));
FUNC F_k(z,kl) = alpha*exp(z)*pow(kl,alpha-1);
FUNC F_L(z,kl) = (1-alpha)*exp(z)*pow(kl,alpha);
FUNC util(c,H) = log(c) - eta*H;
```

The next part describes the optimization problem of the economic agent as a dynamic programming problem.

```
DYNPROG hhprob(Kbeg,iDSunused,iProdtty,Kend,iH);
  DISC = beta;  # expression for discount factor (need not be constant)
  PREP:
  SET h = Hours[iH];
  SET CashoH = (1+r[t])*Kbeg + wage[t]*indProdtty[iProdtty]*h; # aggregate vars have a tim

  IEND = 1;   # expression defining end-of-period discrete state; here identical with disc
  RANGE:
  cclower = MINGRID; # lower bound of choice variable (Kend here)
  ccupper = min(CashoH,MAXGRID) - 1e-10;  # upper bound of choice variable (Kend here)
  return INTERVAL(cclower,ccupper);
  UTIL:
  cons = CashoH-Kend;
  assert(cons>0)
  return util(cons,h); # current utility
  TEXOG = TransProdtty;  # which matrix to use for transition function of exogenous state
  NDC = 2;  # the number of discrete choices
END;
```

The line

```
DYNPROG hhprob(Kbeg,iDSunused,iProdtty,Kend,iH);
```

says that the name of the problem is "hhprob", and it gives the name of the 5 input arguments to the functions of this problem: "Kbeg" is the name of the continuous state, which is capital

at the beginning of period t. "iDSunused" is the index of the endogenous discrete state; since there is none here, this name is not used in the following program. "iProdtty" is the index of the discrete exogenous state, which is labor productivity. "Kend" is the continuous choice variable: end-of-period capital. "iH" is the index of the discrete choice variable, which indicates working or not working, cf. below.

The keyword DISC in the line

```
DISC = beta;
```

indicates that beta is the time discount factor. The discount factor need not be a constant, it can be any expression that depends on the discrete state (endogenous or exogenous) of the household.

In the section started with "PREP:", some variables are set that do not depend on the continuous choice Kend. These variables (here hours h and cash-on-hand CashoH) are computed and saved for efficiency reasons.

The keyword "IEND" gives the end-of-period endogenous discrete state. In this example, it always equals 1, since there is no choice here. (For an example with endogenous discrete state, cf. Section **??**.)

The section started by "RANGE:" defines the range of admissible values for the endogenous choice (end-of-period endogenous state). In general (although not in this specifici model) it depends on the state and also on the discrete choice of the agent. There are three distincit possibilities:

1. "INFEASIBLE" means that at this state and with this discrete choice there is no feasible action. This should only happen if there are other discrete choices for which some actions are feasible. Otherwise, this state would get a value of minus infinity.

2. "POINT(x)" means that the only possible choice is x. For example, in a model of state-continent pricing, if the discrete choice means "keep price unchanged", then the only possible end-of-period price is the beginning-of-period price.

3. "INTERVAL(a,b)" means that the interval (a, b) is feasible.

This setup is more flexible than it appears. For example, if a certain discrete choice allows exactly two choices of the endogenous action, call them a and b, it has to be declared as two different discrete choices, one with "POINT(a)" and the other one with "POINT(b)" as possible range.

The section starting with "UTIL:" computes and returns the current objective ("utility") as a function of states and actions.

The line

```
TEXOG = TransProdtty;
```

says that the matrix "TransProdtty" defines the Markov chain governing the transition of the exogenous discrete state (here iProdtty). The line

```
NDC = 2;
```

says that there are 2 discrete choices in this model (working and not working).

Notice that the meaning of the discrete choice iH is defined by its entering the cash-on-hand variable through the expression "Hours[iH]". Since we have defined

```
PAR{1} Hours = [0.0,hbar];
```

the choice iH=1 means not working, iH=2 means working.

The next section of the program defines the model equations.

```
##########################################################################
# MODEL EQUATIONS:
##########################################################################
# MODEL SECTION
# syntax of equations is either of the following:
# variable(t) := expression   [then the name of the equation equals name of the variable]
# equname:  expr1 = expr2   [equname is then the name of the equation]
MODEL;
DYNPROG(hhprob,V,D); # tie hhprob to variables V (value function) and D (distribution)
# TFP:
z[t] &=   rho_aggr*z[t-1] + epsilon[t];
# marginal product of capital
mpk:      r[t] + delta &=  F_k(z[t],KLratio[t]);
# marginal product of labor
mpl:      wage[t] &= (F_L(z[t],KLratio[t]));
KLratio[t] &= K[t]/L[t];
K[t] &= CSMEAN(hhprob,D[t-1],(.)->Kbeg,t); # aggregate capital
KEnd[t] &= CSMEAN(hhprob,D[t-1],(.)->Kend,t); # checking: end of period capital
H[t] &= CSMEAN(hhprob,D[t-1],(.)->Hours[iH],t);
L[t]  &= CSMEAN(hhprob,D[t-1],(.)->Hours[iH]*indProdtty[iProdtty],t);
GDP[t] &= F(z[t],K[t],L[t]);
Inv[t] &= KEnd[t] - (1-delta)*K[t] ;
C[t] &= GDP[t]  - Inv[t];
```

Two types of commands here are special.

```
DYNPROG(hhprob,V,D);
```

says that the the value function and cross-sectional distribution that are related to the problem "hhprob" are the variables "V" and "D" (the names of these variables can be freely chosen, therefore one needs this declaration here). The corresponding vectors of model equations are then called "hhprob_v" and "hhprob_d", but they are only used internally in the toolkit and are not referenced by the user.

The line

```
K[t] &= CSMEAN(hhprob,D[t-1],(.)->Kbeg,t); # aggregate capital
```

means that $K[t]$ is the cross-sectional mean at time t over the distribution $D[t-1]$ (distribution at the end of period $t-1$) of the variable Kend. The expression

```
(.)->Kbeg
```

stands for a function (here Kend) of all the arguments of the problem hhprob.

The next section computes the steady state.

```
##############################################################
# compute steady state
##############################################################
GUESS;
# set guess for steady state solution
# interest rate r is the variable to solve for numerically in the outer loop
r = 0.013 IN [ 0.012,1/beta-1.004];

STST;
!z;  # solve linearly for z, using equation "z"
BISECT(KLratio:mpk,0.001,1000);  # solve nonlinearly for KLratio in interval [0.001,1000],
LIN(wage:mpl); # solve linearly for wage, using equation "mpl"

# SOLVE  STST OF DYNAMIC PROGRAMM
# 1) define the grids (value and distribituion)
upperBoundK = KLratio(stst)*hbar*10; # upper bound
# make grid equidistant in log(K+shift):
# grid for value function:
dxAtLowerbound = indProdtty[1]*wage(stst)*hbar/8;
gridV = logspaceshiftAtA(lowerBoundK,upperBoundK,nGridV,dxAtLowerbound);
# grid for cross-sectional distribution of capital:
dxAtLowerbound = indProdtty[1]*wage(stst)*hbar/20;
gridD =  logspaceshiftAtA(lowerBoundK,upperBoundK,nGridD,dxAtLowerbound);
# define grids for hhprob, and solve it:
DYNPROG(hhprob,gridV,gridD);
```

```
LIN(K);
LIN(KEnd);
LIN(H);
LIN(L);
LIN(GDP);
LIN(Inv);
LIN(C);
# KLratio is the equation to be satisfied to find GUESS r
RESID KLratio;
```

The line

```
DYNPROG(hhprob,gridV,gridD);
```

does two things:

1. It says that the capital grid on which the value function is defined is called "gridV", and the grid on which the distribution is defined is called "gridD". These variables must have been defined before in the steady state section. In principle, they could have also been defined as parameter vectors in the parameter section, but in general they will depend on other steady state variables, so they will usually be defined in the steady state section.

2. It is a commend to solve for the steady state values of the value function and the cross-sectional distribution. This command just therefore be palced *after* all the variables which enter the agent problem (here: wage and interest rate) are set.

To summarize, the "DYNPROG" command appears at three places in the program:

1. Before the model section, it defines the dynamic programming problem and gives it a name (here: "hhprob").

2. In the model section, it links the value and distribution variables to this name, and implicitly declares the equations of this problem in the model.

3. In the steady state section, it declares the value function and distribution grids to the dynamic programming problem, and indicates that it should now be solved solved in steady state.

8.3 Introducing an endogenous discrete state

Let us assume a cost of switching between employment states, captured by the function switchcost(iHlast,iH). We then change the definition of the dynamic program to

```
DYNPROG hhprob(Kbeg,iHlast,iProdtty,Kend,iH);
  DISC = beta;
  PREP:
  SET h = Hours[iH];
  SET CashoH = (1+r[t])*Kbeg + wage[t]*indProdtty[iProdtty]*h;
  IEND = iH;
  RANGE:
  cclower = MINGRID;
  ccupper = min(CashoH,MAXGRID);
  return INTERVAL(cclower,ccupper);
  UTIL:
  cons = CashoH-Kend;
  return util(cons,h,eta)-switchcost(iHlast,iH);
  TEXOG = TransExog;
  NDC = 2;
END;
```

The discrete state has now two components, the endogenous and the exogenous one. The employment state has no stochastic component, so the transition matrix is the identity, and the total transition matrix is the Kronecker product of the two transition matrices:

```
(TransProdtty,logY)= markovappr(rho,sigma,2.5,nY);
TransEmpl = eye(2);
PAR{2} TransExog = kron(TransProdtty,TransEmpl);
```

This is captured in the line

```
  TEXOG = TransExog;
```

The endogenous discrete state has two realizations, which is the second dimension of the value function and the distribution:

```
VAR{3} D(nGrid-1,2,nY);
VAR{3} V(nK,2,nY);
```

9 Global nonlinear solution

To solve the model nonlinearly, one needs appropriate declarations in the het-file, and the computation of the solution furthermore requires a Julia script detailing the relevant choices.

9.1 Nonlinear section in het-file

Take as example the RBC model:

```
NONLIN;
PREDET(k:Inv);
STATE(z);
APPROX(c:euler);
APPROX(L:ls);
DEFINE(y);
DEFINE(Inv:c);
DEFINE(w);
DEFINE(r:mpc);
```

The command

```
PREDET(k:Inv);
```

declares k as a predetermined variable, this means that $k[t-1]$ is a predetermined state in period t. A state needs a transtion equation, which here is given by the equation Inv.

The command

```
STATE(z);
```

declares $z[t]$ as a state variable in period t, with transition equation also called z.

The command

```
APPROX(c:euler);
```

says that the variable c should be approximated as a function of the states (for example by polynomials, to be determined in an extra Julia script). In a weighted residual method, the residuals of the equation "euler" are used to find the parameters of the polynomial approximation.

The command

```
APPROX(L:ls);
```

does the same thing for L with equation "ls".

The command

```
DEFINE(y);
```

states that the variable y is defined as an analytic expression in all the other variables so far defined, using the equation named "y". This requires that equation "y" can be solved linearly for y, and that all the variables appearing in "y" have already been defined before.

The commands

```
DEFINE(Inv:c);
DEFINE(w);
DEFINE(r:mpc);
```

do the same thing for variables Inv, w and r using equations "c", "w" and "mpc", respectively.

9.2 Julia script

Basic solution and analysis is performed in the following script (file "dononl.jl"):

```
# compute linearized solution first:
rbclin = @linsol rbcnonl

# define shock process:
(x,p) = qnwnorm1(51,0.,0.007);  # approximate normal distribution with standard deviation
shock1 = DiscreteIID(x,p); # make it a type DiscreteIID
xp = (shock1,)  # transform to Tuple
# define nonlinear model:
rbcnonl = defmodelnl(rbclin,xp);

# define approximation:
fspace = PolyAppr(2,8);  # degree-8 polynominal in 2 dimensions (2 states)
(fspace,coef) = iterNonl2(rbcnonl,fspace,xp); # solve iteratively

# check Accuracy:
Grid = randomGrid(fspace,20.0);
res = residnl(_nl,Grid,fspace,coef,xp)
statsResid(res)


# do a simulation:
x0 = stateDetermStSt(rbcnonl);  # starting point: deterministic steady state
sim = simulnl(_nl,x0,fspace,coef,xp,rand(1,400));

# policy function: vary state variable 2  around steady state;
pol = policynl(gridVaryState(fspace,2,101), 1, _pars,_varsStSt,fspace,coef,xp);
plot(pol[:z],pol[:k]);
```